

El juez automático: el primer envío

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Agosto 2026

Objetivos de aprendizaje I

Al terminar este material, el estudiante podrá

- Entrar a la arena del curso, abrir un enunciado y enviar una solución.
- Leer un enunciado tipo juez separando el formato de entrada del formato de salida.
- Resolver *The $3n + 1$ problem* en C o C++ leyendo hasta el final del archivo.
- Probar contra los ejemplos oficiales antes de gastar un envío.
- Interpretar los veredictos y decidir qué revisar con cada uno.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA3: describir ideas con argumentos precisos y basados en evidencia; el material técnico está en inglés.

Cómo usar este material

Es para revisar por cuenta propia antes de la primera tarea. El recorrido se puede seguir con el navegador abierto al lado.

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código
- 5 Probar antes de enviar
- 6 Los veredictos
- 7 Lo que está en manos del estudiante
- 8 Referencias

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código
- 5 Probar antes de enviar
- 6 Los veredictos
- 7 Lo que está en manos del estudiante
- 8 Referencias

Una pregunta para abrir l

Su programa imprime la respuesta correcta con el ejemplo del enunciado.

¿Cómo sabemos que también la imprime con los datos que usted nunca vio?

Lo que hace el servidor I

Cuando usted envía un archivo fuente, el servidor:

- 1 Lo compila con la misma línea de compilación para todos.
- 2 Lo ejecuta con varios juegos de datos que usted no conoce.
- 3 Le da a cada ejecución un tope de tiempo y de memoria.
- 4 Compara la salida producida con la salida esperada.
- 5 Devuelve un veredicto.

El punto que más cuesta al principio

La comparación es entre textos, no entre ideas. El servidor lee las dos salidas palabra por palabra: un saludo, un rótulo o un número de más cuentan como respuesta incorrecta, así el cálculo esté bien hecho.

El trato es con la entrada estándar I

Contrato de todo problema tipo juez

El programa lee de la entrada estándar y escribe en la salida estándar. No pide datos, no muestra menús, no lee archivos y no espera que alguien esté mirando la pantalla.

Lo que ocurre en la práctica

El servidor conecta un archivo a la entrada de su programa y guarda en otro archivo lo que su programa escriba. Es lo mismo que usted hace en la terminal con `./sol < entrada.txt > salida.txt`.

Dos costumbres que hay que desaprender I

Programa de clase

- Saluda al usuario.
- Pide el dato: "Ingrese n".
- Explica el resultado.
- Resuelve un caso y termina.

Programa para el juez

- No saluda.
- Lee el dato sin anunciarlo.
- Escribe solo el resultado.
- Resuelve todos los casos que vengan.

El servidor del curso es DOMjudge. La interfaz de equipo está en <http://arena.javerianacali.edu.co/domjudge/team>.

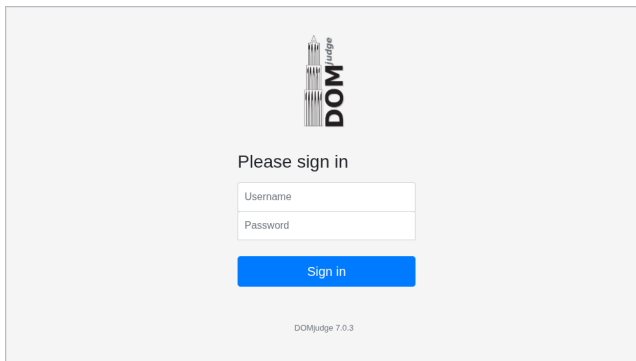
Dos detalles de la dirección

Va con `http`, no con `https`, y termina en `/domjudge/team`. El marcador público está en `/domjudge/public`.

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código
- 5 Probar antes de enviar
- 6 Los veredictos
- 7 Lo que está en manos del estudiante
- 8 Referencias

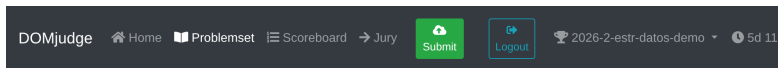
Paso 1: entrar I



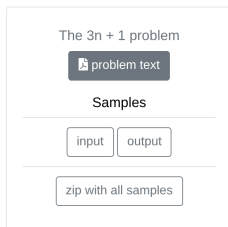
The image shows a login interface for DOMjudge. At the top center is the DOMjudge logo, which consists of a stylized vertical stack of books or papers with the word "DOMjudge" written vertically to its right. Below the logo, the text "Please sign in" is displayed. Underneath this text are two input fields: the first is labeled "Username" and the second is labeled "Password". Below these fields is a blue button with the text "Sign in" in white. At the bottom center of the interface, the version number "DOMjudge 7.0.3" is displayed.

Desde la portada del servidor se sigue el enlace *Team interface*; ahí se escriben las credenciales del curso. Si la página no carga, conviene revisar que la dirección vaya con `http` y que no haya una VPN comercial encendida.

Paso 2: abrir el enunciado I



Contest problems



En *Problemset* está la lista de problemas del concurso. El botón *problem text* abre el PDF; *input* y *output* descargan el ejemplo oficial, y *zip with all samples* los trae juntos.

Paso 3: leer el enunciado I

Estructuras de Datos Diseño de la arena 2020-2

The $3n + 1$ problem

Source file name: 2n1.c

Time limit: 3 seconds

Consider the following algorithm applied to a positive integer n : if n is 1, the algorithm stops; if n is even, n is replaced by $n/2$; otherwise, n is replaced by $3n + 1$. It is conjectured — but not yet proved — that the algorithm terminates for every starting value.

The cycle length of n is the amount of numbers that the algorithm visits, including n itself and the final 1. For example, starting at 22 the algorithm visits

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

so the cycle length of 22 is 16.

For every pair of numbers i and j , determine the maximum cycle length over all numbers between and including i and j .

Input

The input consists of a sequence of pairs of integers i and j ($0 < i, j < 1\,000\,000$), one pair per line. Process every pair until the end of the input. Note that the intermediate values of the sequence may exceed the capacity of a 32-bit integer: use a 64-bit type.

The input must be read from standard input.

Output

For each pair print a single line with i , j and the maximum cycle length for the integers between and including i and j , separated by single spaces. The numbers i and j must appear in the output in the same order in which they appear in the input.

The output must be written to standard output.

Sample Input

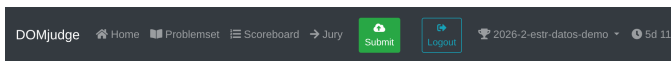
1 10
100 200
201 210
900 1000

Sample Output

1 10 20
100 200 128
201 210 89
900 1000 174

Paso 4: enviar I

Se sube el archivo fuente, no el ejecutable ni un comprimido, y se revisa que el problema y el lenguaje sean los que corresponden.



Submit

Source files

No file selected

Browse

Problem

Select a problem

Language

Select a language

Submit

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema**
- 4 De la idea al código
- 5 Probar antes de enviar
- 6 Los veredictos
- 7 Lo que está en manos del estudiante
- 8 Referencias

The $3n + 1$ problem I

Statement

Consider the following algorithm applied to a positive integer n : if n is 1, the algorithm stops; if n is even, n is replaced by $n/2$; otherwise, n is replaced by $3n + 1$.

The cycle length of n is the amount of numbers that the algorithm visits, including n itself and the final 1.

For every pair of numbers i and j , determine the maximum cycle length over all numbers between and including i and j .

Datos de la ficha

Archivo fuente: 3n1.c. Tope de tiempo: 3 segundos. Cotas:
 $0 < i, j < 1\,000\,000$.

La longitud de ciclo, con un ejemplo I

Partiendo de 22, el algoritmo visita:

22 11 34 17 52 26 13 40 20 10 5 16 8 4 2 1

Son 16 números contando el 22 del principio y el 1 del final, así que la longitud de ciclo de 22 es 16.

Los primeros pasos, uno por uno

22 es par, se parte: 11. 11 es impar, $3 \cdot 11 + 1 = 34$. 34 es par: 17. 17 es impar: 52. Y así hasta llegar a 1.

Formatos de entrada y salida I

Input

The input consists of a sequence of pairs of integers i and j , one pair per line. Process every pair until the end of the input.

Output

For each pair print a single line with i , j and the maximum cycle length, separated by single spaces. The numbers i and j must appear in the output in the same order in which they appear in the input.

El ejemplo oficial I

Sample input

```
1 10
100 200
201 210
900 1000
```

Sample output

```
1 10 20
100 200 125
201 210 89
900 1000 174
```

Cuatro pares de entrada, cuatro líneas de salida. La correspondencia es uno a uno.

¿Entendimos el problema? I

En una frase

Por cada par de números, recorrer todo el intervalo que delimitan y reportar la longitud de ciclo más grande que aparezca.

Entradas

- Pares de enteros, uno por línea.
- Cuántos pares hay no se anuncia.
- Se acaban con el archivo.

Salidas

- Una línea por par.
- Tres números separados por un espacio.
- i y j en el orden en que llegaron.

Las dos trampas del enunciado I

El orden del par

Nada obliga a que i sea menor que j . El intervalo hay que recorrerlo del menor al mayor, pero al imprimir van i y j como llegaron. Con la entrada 10 1 la respuesta es 10 1 20, no 1 10 20 ni 10 1 0.

El tamaño de los números

Aunque i y j se quedan por debajo de un millón, los valores intermedios de la sucesión suben mucho más. El enunciado lo advierte: hay que usar un tipo de 64 bits, `long long` en C.

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código**
- 5 Probar antes de enviar
- 6 Los veredictos
- 7 Lo que está en manos del estudiante
- 8 Referencias

Cómo sabe el programa que se acabó la entrada l

En C

`scanf` devuelve cuántos valores logró leer. Mientras consiga los dos enteros del par devuelve 2; al llegar al final del archivo devuelve EOF.

En C++

La lectura `std::cin >> i >> j` devuelve el flujo, y el flujo se evalúa como falso cuando ya no hay nada que leer.

En la terminal

Al probar a mano, el final del archivo se produce con `Ctrl+D` en Linux y macOS, o `Ctrl+Z` seguido de `Enter` en Windows.

La función que cuenta el ciclo I

```
/* Cuantos numeros visita el algoritmo desde n hasta  
   llegar a 1, contando los dos extremos. */  
long long longitud(long long n)  
{  
    long long cuenta;  
  
    cuenta = 1;  
    while (n > 1)  
    {  
        if (n % 2 == 0)  
        {  
            n = n / 2;  
        }  
        else  
        {  
            n = 3 * n + 1;  
        }  
        cuenta = cuenta + 1;  
    }  
  
    return cuenta;  
}
```


La traza de longitud(22) I

Vuelta	n al entrar	Regla aplicada
1	22	par, se parte $\rightarrow 11$
2	11	impar, $3n + 1 \rightarrow 34$
3	34	par $\rightarrow 17$
4	17	impar $\rightarrow 52$
$\vdots$	$\vdots$	$\vdots$
15	2	par $\rightarrow 1$
—	1	la condición falla, se sale

cuenta arranca en 1 y sube una vez por vuelta: 15 vueltas más el 1 inicial dan 16.

El programa principal: leer y ordenar I

```
int main(void)
{
    long long i, j, desde, hasta, n, mejor, actual;

    while (scanf("%lld %lld", &i, &j) == 2)
    {
        /* El intervalo se recorre de menor a mayor;
           i y j se imprimen como llegaron. */
        desde = i;
        hasta = j;
        if (desde > hasta)
        {
            desde = j;
            hasta = i;
        }
    }
}
```

El programa principal: barrer e imprimir I

```
mejor = 0;
n = desde;
while (n <= hasta)
{
    actual = longitud(n);
    if (actual > mejor)
    {
        mejor = actual;
    }
    n = n + 1;
}

printf("%lld %lld %lld\n", i, j, mejor);
}

return 0;
}
```

La versión en C++ I

Misma idea; cambian la lectura y la escritura.

```
int main()
{
    long long i, j, desde, hasta, n, mejor, actual;

    while (std::cin >> i >> j)
    {
        desde = i;
        hasta = j;
        if (desde > hasta)
        {
            desde = j;
            hasta = i;
        }

        mejor = 0;
        n = desde;
```

La versión en C++: la escritura I

El barrido del intervalo es idéntico al de C. Lo único que cambia al final es cómo se imprime:

```
std::cout << i << " " << j << " "  
          << mejor << "\n";
```

El " " entre los números es un espacio, y el `\n` del final cierra la línea. El enunciado pide exactamente eso: tres números separados por un solo espacio, uno por línea.

Costo de la solución I

Tiempo

Por cada par se recorren $|j - i| + 1$ números, y por cada número se sigue su cadena hasta llegar a 1. Si L es la longitud de cadena más larga del intervalo, el costo de un par es $O((|j - i| + 1) \cdot L)$.

Espacio

Se usan unas pocas variables, sin importar el tamaño de la entrada: $O(1)$.

Medido

El intervalo completo, 1 999999, se resuelve en unos 0,2 segundos. El tope del problema es 3 segundos, así que la solución directa alcanza sin necesidad de guardar resultados intermedios.

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código
- 5 Probar antes de enviar**
- 6 Los veredictos
- 7 Lo que está en manos del estudiante
- 8 Referencias

El ciclo de trabajo local I

- 1 Descargar *input* y *output* desde la página del problema.
- 2 Compilar con las advertencias encendidas.
- 3 Ejecutar redirigiendo la entrada.
- 4 Comparar con `diff`.

Los tres comandos

```
gcc -Wall -Wextra 3n1.c -o sol  
./sol < 3n1.in > salida.out  
diff salida.out 3n1.out
```

Si `diff` no imprime nada, las dos salidas son idénticas. Esa es la señal para enviar.

Leer la salida de diff |

Con una solución que se equivoca, diff responde así:

```
1c1
< 10 1 0
---
> 10 1 20
```

Cómo se lee

1c1 dice que la línea 1 del primer archivo hay que cambiarla por la línea 1 del segundo. Las líneas con < son las que produjo el programa; las que llevan >, las que se esperaban.

Cuando diff no dice nada I

```
$ ./sol < 3n1.in > salida.out  
$ diff salida.out 3n1.out  
$ echo $?  
0
```

El silencio es la respuesta buena: los dos archivos son idénticos byte por byte. El 0 de echo `$?` lo confirma sin ambigüedad.

Que diff calle con el ejemplo del enunciado no garantiza el ACCEPTED: los juegos de datos del servidor son otros y más grandes.

Probar con datos propios I

El ejemplo oficial es el caso amable: en sus cuatro líneas, i siempre es menor que j . Vale la pena agregar a mano:

- un par al revés, como 10 1;
- un par de un solo número, como 22 22;
- el intervalo más grande que permita el enunciado;
- una entrada vacía, que debe producir salida vacía sin quedarse colgado.

Los juegos de datos ocultos incluyen los extremos. Un programa que solo se probó con el ejemplo del enunciado llega al servidor sin haber visto sus casos difíciles.

Ver los caracteres que no se ven I

Cuando el diff señala una diferencia invisible

Un espacio al final de la línea o un salto de línea faltante no se distinguen a simple vista. El comando `cat -A` los muestra: marca el final de cada línea con `$` y los tabuladores con `^I`.

```
./sol < 3n1.in | cat -A | head -3
```

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código
- 5 Probar antes de enviar
- 6 Los veredictos**
- 7 Lo que está en manos del estudiante
- 8 Referencias

Tres envíos del mismo problema I

Los tres compilaron sin errores y los tres calculan longitudes de ciclo. Solo uno pasó.

DOMjudge

[Home](#) [Problemset](#) [Scoreboard](#) [Jury](#)

[Submit](#) [Logout](#)

2026-2-estr-datos-demo 5d 11h

RANK	TEAM	SCORE	3n1
1	Carlos Delgado	1 9	-11 2 hrs

Submissions			
time	problem	lang	result
13:48	3n1	c	TIMELIMIT
13:47	3n1	c	WRONG-ANSWER
13:44	3n1	c	CORRECT
22:49	3n1	c	CORRECT
22:39	3n1	c	CORRECT
22:39	3n1	c	WRONG-ANSWER

Clarifications

No clarifications.

Clarification Requests

No clarification request.

request clarification

El que dio WRONG-ANSWER I

Esta versión supone que i siempre viene antes que j .

```
int main(void)
{
    long long i, j, n, mejor, actual;

    while (scanf("%lld %lld", &i, &j) == 2)
    {
        mejor = 0;
        n = i;
        while (n <= j)
        {
            actual = longitud(n);
            if (actual > mejor)
            {
                mejor = actual;
            }
            n = n + 1;
        }

        printf("%lld %lld %lld\n", i, j, mejor);
    }
}
```

Dónde falla I

Con el ejemplo del enunciado da bien

Sus cuatro pares vienen en orden creciente, así que el programa hace exactamente lo mismo que la solución buena.

Con un par al revés se cae

Cuando llega $10\ 1$, la condición $n \leq j$ es falsa desde el principio: el ciclo no entra nunca, mejor se queda en 0 y el programa imprime $10\ 1\ 0$ en lugar de $10\ 1\ 20$. El servidor sí prueba pares al revés, y por eso devolvió WRONG-ANSWER.

El que dio TIMELIMIT I

Esta otra sí ordena el par, pero por cada consulta arranca el recorrido en 1 en lugar de arrancar en el extremo menor.

```
mejor = 0;
n = 1;
while (n <= hasta)
{
    actual = longitud(n);
    if (n >= desde && actual > mejor)
    {
        mejor = actual;
    }
    n = n + 1;
}

printf("%lld %lld %lld\n", i, j, mejor);
```

Por qué el juez la rechaza I

Las respuestas son correctas

El filtro $n \geq$ desde descarta lo que sobra, así que imprime exactamente lo mismo que la solución buena. El problema no es qué calcula sino cuánto le cuesta.

El costo cambió de lugar

La versión buena recorre $|j - i| + 1$ números; esta recorre siempre desde 1 hasta el mayor de los dos. Con cien pares estrechos cerca del millón, la buena tarda cinco milésimas y esta tarda casi 20 segundos. El tope es de 3.

Qué dice ese veredicto I

TIMELIMIT no señala un error de lógica sino uno de costo. Revisar la salida no sirve: hay que contar cuántas operaciones hace el programa, que es el análisis de complejidad del curso.

La pregunta que hay que hacerse

¿Cuántas veces se ejecuta el ciclo más interno en el peor caso? Si la respuesta crece con el tamaño de la entrada más rápido de lo que crece el problema, sobra trabajo en alguna parte.

Los veredictos y qué revisar I

Veredicto	Qué revisar
CORRECT	Nada. La solución pasó todos los juegos de datos.
WRONG-ANSWER	El formato de salida y los casos límite; el desbordamiento de los tipos.
TIMELIMIT	El costo del algoritmo, o un ciclo que no termina cuando se acaba la entrada.
RUN-ERROR	Índices fuera de rango, división entre cero, memoria no reservada, o un <code>return</code> distinto de cero.
COMPILER-ERROR	El código no compila con la línea del servidor; conviene compilar en limpio antes de enviar.

El marcador I

RANK	TEAM	SCORE	3n1
1	Carlos Delgado	1 9	-11 2 tries
2	 	0 0	
	 	0 0	
	 	0 0	
	 	0 0	
	 	0 0	
	 	0 0	
	 	0 0	
	 	0 0	
	 	0 0	

Los envíos fallidos quedan registrados y suman penalización.

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código
- 5 Probar antes de enviar
- 6 Los veredictos
- 7 Lo que está en manos del estudiante**
- 8 Referencias

La lista antes de oprimir Submit I

- 1 El programa no imprime nada que el enunciado no pida.
- 2 Lee hasta el final del archivo, no un solo caso.
- 3 Compila con `-Wall -Wextra` y sin advertencias.
- 4 `diff` contra el ejemplo oficial no dice nada.
- 5 Se probaron el par al revés, el intervalo de un solo número y el más grande que permita el enunciado.
- 6 Los tipos aguantan los valores intermedios.
- 7 En el formulario, el problema y el lenguaje son los que corresponden.

De quién es la responsabilidad I

Verificar la solución antes de enviarla es parte del trabajo, no un paso opcional. Un envío que falla por no haber probado —salida con texto de más, un solo caso leído, un tipo que se desborda, un ciclo que no termina— corre por cuenta de quien lo envió.

Lo que no se acepta como reclamo

Que el programa “funcionaba en mi máquina”, que “el ejemplo del enunciado daba bien” o que “se subió el archivo equivocado”. Los tres se evitan con la lista de la lámina anterior, y los tres se revisan en menos de un minuto.

Lo que sí se atiende I

Un problema del servidor, un enunciado ambiguo o un juego de datos que contradiga lo enunciado. Para eso está el botón *request clarification* de la interfaz de equipo, que deja la pregunta registrada y con respuesta visible.

Antes de escribir la solicitud

Conviene releer el formato de salida y probar el caso que se cree contradictorio. La mayoría de las dudas se resuelven ahí mismo.

Resumen operativo I

- 1 Leer los formatos de entrada y salida antes que la descripción.
- 2 Descargar el ejemplo oficial desde la página del problema.
- 3 Escribir el programa sin diálogo, leyendo hasta el final del archivo.
- 4 Compilar limpio, ejecutar redirigido y comparar con `diff`.
- 5 Agregar los casos límite que el ejemplo no trae.
- 6 Enviar el archivo fuente y leer el veredicto.

Contenido I

- 1 Por qué un juez automático
- 2 El recorrido en el arena
- 3 El problema
- 4 De la idea al código
- 5 Probar antes de enviar
- 6 Los veredictos
- 7 Lo que está en manos del estudiante
- 8 Referencias**

Referencias I

-  R. Thareja, *Data Structures Using C*. Oxford University Press, 2018, cap. 1.
-  N. Kalicharan, *Data Structures in C*. 2008, cap. 1.
-  T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein, *Introduction to Algorithms*, 4.^a ed. MIT Press, 2022, secc. 2.2 y 3.1.
-  DOMjudge Team, *DOMjudge Team Manual*. Disponible en <https://www.domjudge.org/docs/manual/>.